

Experimental Instructions 1

In this experiment, we will try to realize Hadoop data processing in VirtualBox virtual machine environment. We have two jobs: **Filter** out the words and **sort** all words.

• Step One-- Set up the environment

Download Virtual Box and import appliance

1. Download Virtual Box from <https://www.virtualbox.org/wiki/Downloads>

Download VirtualBox

Here you will find links to VirtualBox binaries and its source code.

VirtualBox binaries

By downloading, you agree to the terms and conditions of the respective license.

If you're looking for the latest VirtualBox 6.0 packages, see [VirtualBox 6.0 builds](#). Please also use version 6.0

If you're looking for the latest VirtualBox 5.2 packages, see [VirtualBox 5.2 builds](#). Please also use version 5.2

VirtualBox 6.1.16 platform packages

- [Windows hosts](#)
- [OS X hosts](#)
- [Linux distributions](#)
- [Solaris hosts](#)

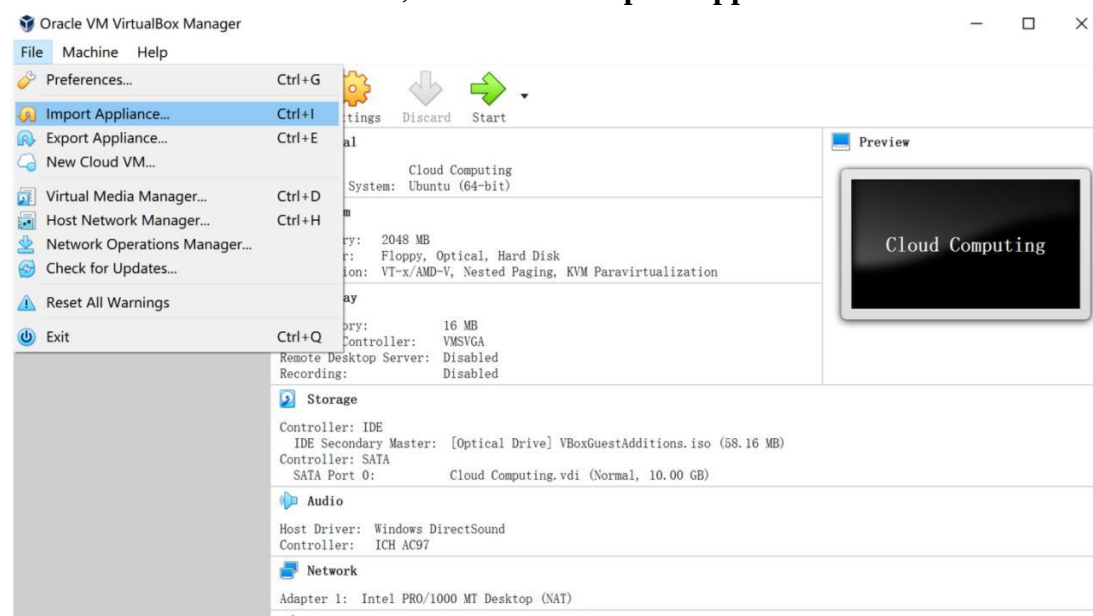
The binaries are released under the terms of the GPL version 2.

See the [changelog](#) for what has changed.

You might want to compare the checksums to verify the integrity of downloaded packages. *The SHA256 check*

- [SHA256 checksums](#), [MD5 checksums](#)

2. Install and start Virtual Box, click **File -> Import Appliance**



3. Select the given ova file and click next to import the appliance

← Import Virtual Appliance

Appliance to import

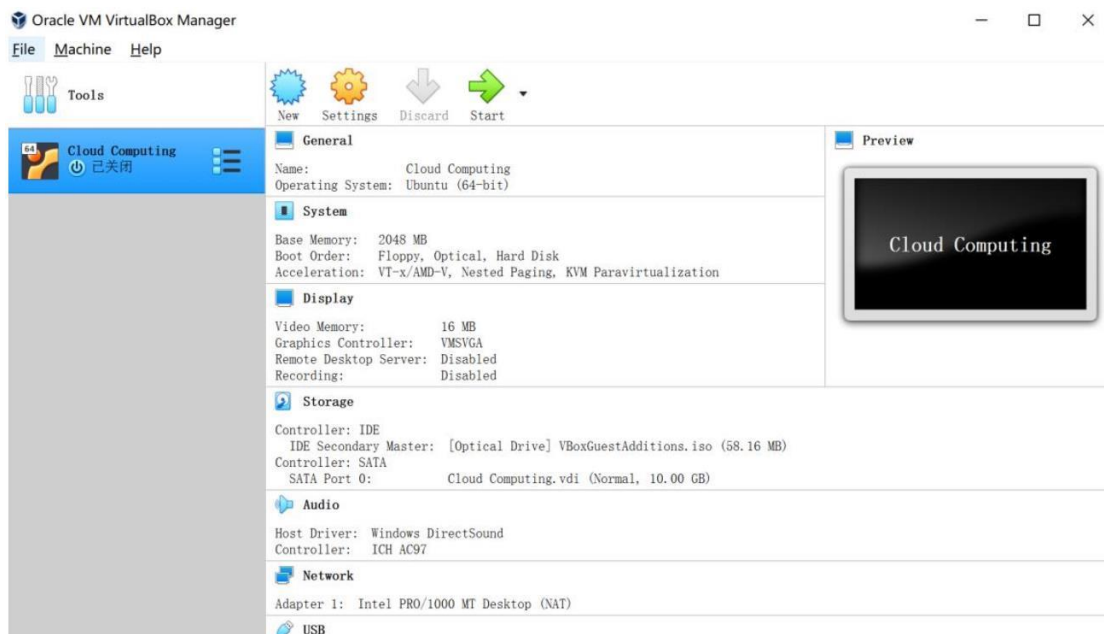
Please choose the source to import appliance from. This can be a local file system to import OVF archive or one of known cloud service providers to import cloud VM from.

Source: Local File System

Please choose a file to import the virtual appliance from. VirtualBox currently supports importing appliances saved in the Open Virtualization Format (OVF). To continue, select the file to import below.

File: C:\Users\lenovo\Documents\Cloud Computing.ova

4. Click **Start** to start the system



5. Login with **username: root, password: cloud2021**

```
Ubuntu 16.04.6 LTS ubuntu tty1
ubuntu login: root
Password:
Last login: Fri Dec  3 11:10:35 CST 2021 on tty1
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-142-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

204 packages can be updated.
138 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

root@ubuntu:~#
```

• Step Two --- Complete the code snippet

Exercises - WordCountAndSort

1. Run the following commands to find the source file

1. `cd /root/cloudz`
2. `ls`

```
root@ubuntu:~# cd /root/cloudz/  
root@ubuntu:~/cloudz# ls  
hadoop-2.7.3  hadoop-2.7.3.tar.gz  WordCountAndSort.java  
root@ubuntu:~/cloudz# _
```

In this exercise (WordCountAndSort.java), there are **two jobs**:
One is to filter and count the word, and the other is to sort the words. The words have been generated by a random text generator and have been placed in the HDFS inside the virtual machine, but there are some messy codes in that file.

2. You are required to implement a counter that comply with the followings:

- **Job1**

1. Filter out the words including other characters not in the alphabet.
2. Make these words uppercased and save the count for them in the output.

- **Job2**

3. Read the outputs of Job1;
4. Sort all words into a descending order of their count.
5. Output results to HDFS.

Notice: All you need to do is to make the necessary changes (marked as **`To Do`**) in the file "WordCountAndSort.java". Don't change the part marked as **'Don't change'**.

• Step Three -- Verification

Compile the source code

1. **Compile** through the following command

```
1. javac WordCountAndSort.java
```

2. **Package** as Jar file

```
1. jar -cvf WordCountAndSort.jar ./WordCountAndSort*.class
```

3. Before you run the job, check if the hadoop service is started, enter `jps`

if started, like this

```
root@ubuntu:~/cloudz# jps
1971 ResourceManager
2100 NodeManager
1622 DataNode
2407 Jps
1817 SecondaryNameNode
1467 NameNode
```

otherwise

```
root@ubuntu:~/cloudz# jps
1306 Jps
root@ubuntu:~/cloudz# _
```

To start the service, enter the command

```
1. /root/cloudz/hadoop-2.7.3/sbin/start-all.sh
```

```
root@ubuntu:~/cloudz# /root/cloudz/hadoop-2.7.3/sbin/start-all.sh
This script is Deprecated. Instead use start-dfs.sh and start-yarn.sh
Starting namenodes on [localhost]
localhost: starting namenode, logging to /root/cloudz/hadoop-2.7.3/logs/hadoop-root-namenode-ubuntu.out
localhost: starting datanode, logging to /root/cloudz/hadoop-2.7.3/logs/hadoop-root-datanode-ubuntu.out
Starting secondary namenodes [0.0.0.0]
0.0.0.0: starting secondarynamenode, logging to /root/cloudz/hadoop-2.7.3/logs/hadoop-root-secondarynamenode-ubuntu.out
starting yarn daemons
starting resourcemanager, logging to /root/cloudz/hadoop-2.7.3/logs/yarn-root-resourcemanager-ubuntu.out
localhost: starting nodemanager, logging to /root/cloudz/hadoop-2.7.3/logs/yarn-root-nodemanager-ubuntu.out
root@ubuntu:~/cloudz#
```

And it will be started after a few seconds, check again

4. Run the application

1. `hadoop jar WordCountAndSort.jar WordCountAndSort /dataset1G /sortResult`

If the job successfully starts, it will look like this

```
INFO client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
INFO input.FileInputFormat: Total input paths to process : 1
INFO mapreduce.JobSubmitter: number of splits:9
INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1511924348280_0001
INFO impl.YarnClientImpl: Submitted application application_1511924348280_0001
INFO mapreduce.Job: The url to track the job: http://localhost:8099/proxy/application_1511924348280_0001
INFO mapreduce.Job: Running job: job_1511924348280_0001
```

5. Example to check the result

1. `hadoop fs -cat /sortResult/part-r-00000 | head`

**** Show your result in the experimental report.**